

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

Отчет по блоку задач №1 (Яндекс.Контест)

Выполнил:

Студент группы Р3209

Евграфов А. А.

Преподаватель:

Косяков М. С.

Санкт-Петербург, 2026 г.

Задачи из 1-го блока Яндекс.Контеста

Задача №1 «А. Агроном-любитель»

Описание

Лёша вырастил n цветов в ряд. Требуется найти самый длинный непрерывный участок, в котором нет трёх одинаковых цветков подряд. Если таких участков несколько, выбрать самый левый.

Ограничение времени: 2 секунды.

Ограничение памяти: 64 Мб.

Решение

Двигаемся правым указателем r и поддерживаем инвариант: l - наименьшее такое число, что в отрезке $[l, r]$ нет трех подряд идущих цветов (r фиксировано). Если на отрезке $[r - 2, r]$ появились три подряд идущие одинаковые цвета, тогда $l = r - 1$. Каждый раз обновляем максимум, если $r - l + 1$ (длина отрезка) строго больше (так как нужен самый левый подотрезок) текущего максимума res , обновляем его и индексы x, y .

Доказательство

Если $n \leq 2$ то ответ очевиден, так как длина отрезка < 3 . База доказана, будет поддерживать инвариант: $[l, r]$ - самый длинный корректный отрезок с концом в r . Пусть для $r - 1$ утверждение доказано, посмотрим на отрезок $[l, r]$:

а) Если трех одинаковых символов не появилось, тогда $[l, r]$ - самый длинный с концом в r , если это не и существует более длинный валидный $[l_0, r]$, тогда $l_0 < l$ (отрезки $[l_0, r - 1]$ и $[l, r - 1]$ все еще очевидно валидны) - противоречие с шагом индукции при $r - 1$.

б) Если $arr[r] = arr[r - 1] = arr[r - 2]$, тогда самый длинный валидный отрезок с концом в r это $[r - 1, r]$ и $l = r - 1$.

Алгоритм выдаст именно самый левый отрезок, поскольку неравенство строгое.

Алгоритмическая сложность

- По времени: $O(n)$
- По памяти: $O(1)$

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     int n;
10    cin >> n;
11
12    int l = 0;
13    int res = 2;
```

```

14  int x = 0;
15  int y = 1;
16
17  vector<int> arr(n);
18
19  for (int i = 0; i < n; i++) {
20      cin >> arr[i];
21  }
22
23  if (n == 2) {
24      cout << "1 2";
25      return 0;
26  }
27
28  if (n == 1) {
29      cout << "1 1";
30      return 0;
31  }
32
33  for (int r = 2; r < n; r++) {
34      if (arr[r] == arr[r - 1] && arr[r - 1] == arr[r - 2]) {
35          l = r - 1;
36      }
37      if (r - l + 1 > res) {
38          res = r - l + 1;
39          x = l;
40          y = r;
41      }
42  }
43
44  cout << x + 1 << " " << y + 1;
45  return 0;
46  }

```

Задача №2 «В. Зоопарк Глеба»

Описание

На окружности расположены животные (маленькие буквы) и ловушки (большие буквы) в некотором порядке. Требуется определить, можно ли сопоставить каждое животное с ловушкой того же вида так, чтобы пути (хорды) не пересекались. Если да, то для каждой ловушки в порядке обхода вывести номер животного.

Ограничение времени: 1 секунда.

Ограничение памяти: 256 Мб.

Решение

Пусть каждому животному соответствует открывающая скобка, а ловушке - закрывающая. Заметим, что решение существует тогда и только тогда, когда скобочная последовательность является правильной.

В векторе $v1$ храним для каждого животного храним его номер (каким по счёту оно пришло). Аналогично для ловушек в $v2$. В стеке храним еще не обработанные ловушки/животные. Если на вершущке стека и в $s[i]$ одна буква (но разные регистры), то сохраняем индекс обрабатываемого животного в ответ (под индексом имеется в виду номер этого животного в инпуте). Если нет - кладем на стек. В конце работы алгоритма стек должен быть пустой (обработали все животные-ловушки).

Доказательство

Пусть у нас есть граф G с $2n$ вершинами, причем он является простым циклом с n диагоналями (у каждой вершины степень 3). Ответ YES (Possible) тогда и только тогда, когда граф является планарным.

Давайте по индукции (по размеру графа) докажем, что если ответ YES, то тогда есть пара соседних смежных вершин. База очевидна, пусть для меньших графов утверждение доказано. Граф планарный, удалим ребро AB - граф окажется планарным, есть две соседние смежные вершины XY . Добавим ребро AB , если какая-то вершина (НУО вершина X) окажется между A и B , тогда ребро AB пересекает XY - противоречие с планарностью.

Пусть алгоритм работает на графах с меньшим числом вершин (база очевидна).

а) Пусть ответ YES. Найдем ребро графа G соединяющее две смежные вершины - в момент когда мы будем анализировать вторую вершину сработает условие `if` и соответствующая ей вершины пропадает с вершины стека, значит после обработки этой вершины проверка планарности графа G равносильна проверки планарности меньшего графа, а так как меньший граф тоже является планарным, то алгоритм выдаст YES.

б) Пусть ответ NO, предположим алгоритм выдал YES. Значит стек оказался пуст, а тогда в какой-то момент времени мы нашли смежную пару соседних вершин. Так как на меньшем графе алгоритм отрабатывает корректно, то меньший граф является планарным, значит ребро, соединяющее две соседние вершины, пересекало какое-то другое ребро, что невозможно - противоречие.

Алгоритмическая сложность

- По времени: $O(n)$
- По памяти: $O(n)$

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     string s;
10    cin >> s;
11    stack<int> st;
12    vector<int> res(s.size() / 2 + 1);
13    int k = 1;
14    vector<int> v1;
15    vector<int> v2;
16    for (int i = 0; i < (int)s.size(); i++) {
17        if (islower(s[i])) {
18            v1.push_back(k++);
19        } else {
20            v1.push_back(0);
21        }
22    }
23    k = 1;
24    for (int i = 0; i < (int)s.size(); i++) {
25        if (isupper(s[i])) {
26            v2.push_back(k++);
27        } else {
28            v2.push_back(0);
29        }
30    }
31    for (int i = 0; i < (int)s.size(); i++) {
32        if (st.empty()) {
33            st.push(i);
34            continue;
35        }
36
37        bool a1 = (isupper(s[i]) && islower(s[st.top()]));
38        bool a2 = (islower(s[i]) && isupper(s[st.top()]));
39        bool a3 = (tolower(s[i]) == tolower(s[st.top()]));
40
41        if ((a1 || a2) && a3) {
42            if (islower(s[i])) {
43                res[v2[st.top()]] = v1[i];
44            } else {
45                res[v2[i]] = v1[st.top()];
46            }
47            st.pop();
48        } else {
49            st.push(i);
50        }
51    }
52    if (!st.empty()) {
53        cout << "Impossible";
54    } else {
55        cout << "Possible\n";
56        for (int i = 1; i <= (int)s.size() / 2; i++) {
57            cout << res[i] << " ";
58        }
```

```
59 }  
60 return 0;  
61 }
```

Задача №3 «С. Конфигурационный файл»

Описание

Необходимо обработать последовательность строк, представляющих блоки и присваивания переменных. При присваивании одной переменной другой нужно вывести текущее значение второй переменной. Переменные имеют область видимости: внутри блока новые значения действуют только до конца блока.

Ограничение времени: 1 секунда.

Ограничение памяти: 64 Мб.

Решение

Обрабатываем ввод построчно в цикле и поддерживаем инвариант: значение обрабатываемой переменной в текущем блоке является актуальным и лежит в мапе. Когда начинается новый блок на вершину стека кладем фиктивную переменную `mark000` (чтобы понять до куда удалять потом элементы стека) и если мы в новом блоке обрабатываем переменную `var` - кладем ее значение из внешнего блока на стек, обновляем в мапе значение `var`. При выходе из блока возвращаем старые значения из стека в мапу.

Доказательство

Заметим, что скобочки открытия и закрытия блоков образуют ПСП, то есть каждой открывающей скобке соответствует своя закрывающая - пронумеруем все открывающие скобки по порядку их обхода сверху вниз, соответствующим им закрывающим присвоим те же номера. Будем поддерживать инвариант выше: для любой переменной `var` в любой момент времени значение `mp[var]` является актуальным. Рассмотрим произвольный инпут, зафиксируем его. Пусть мы обработали часть строк и инвариант поддерживается (база для одной строки очевидно выполняется). Пусть мы обрабатываем блок $p > 1$ и вложенных в него блоков нет. Когда мы заходим в блок, кладем на стек метку. Когда обрабатываем очередную переменную, кладем на стек ее значение из мапы.

Тогда алгоритм внутри блока работает корректно: все значения были актуальными, если мы в блоке p переменной `var` что-то присваиваем, то ее значение в мапе становится актуальным, если присваиваем ее - значение меняемой переменной также меняется и выводится. Если мы из блока p выходим, то при достижении закрывающей скобки значения измененных переменных будут теми же, которые были на момент вхождения в блок p , поскольку в самом низу стека (после соответствующей метки) лежит значение переменной до ее первого изменения - им перезапишем значение переменной в мапе.

Если внутри блока p есть блок $l = p + 1$, тогда по вышедоказанному после обработки блока l значения в p будут актуальными.

Алгоритмическая сложность

- По времени: $O(N)$, где N - количество строк входа
- По памяти: $O(N)$

Код

```
1 #include <bits/stdc++.h>
2
```

```

3 using namespace std;
4
5 pair<string, string> split(string s) {
6     string s1 = "";
7     string s2 = "";
8     int j = 0;
9     for (int i = 0; i < (int)s.size(); i++) {
10         if (s[i] == '=') {
11             j = i;
12             break;
13         }
14         s1 += s[i];
15     }
16     for (int i = j + 1; i < (int)s.size(); i++) {
17         s2 += s[i];
18     }
19     return make_pair(s1, s2);
20 }
21
22 int main() {
23     ios_base::sync_with_stdio(false);
24     cin.tie(nullptr);
25
26     unordered_map<string, int> mp;
27     stack<pair<string, int>> st;
28     string s;
29     pair<string, int> mark = make_pair("mark000", 0);
30     pair<string, string> p;
31     int k = 0;
32
33     while (cin >> s) {
34         if (s == "{") {
35             st.push(mark);
36             k++;
37         } else if (s == "}") {
38             while (st.top() != mark) {
39                 mp[st.top().first] = st.top().second;
40                 st.pop();
41             }
42             st.pop();
43             k--;
44         } else {
45             p = split(s);
46             if (k > 0) {
47                 st.push(make_pair(p.first, mp[p.first]));
48             }
49             if (p.second[0] >= '0' && p.second[0] <= '9') {
50                 mp[p.first] = stoi(p.second);
51             } else if (p.second[0] == '-') {
52                 mp[p.first] = stoi(p.second);
53             } else {
54                 mp[p.first] = mp[p.second];
55                 cout << mp[p.first] << "\n";
56             }
57         }
58     }
59
60     return 0;
61 }

```

Задача №4 «D. Профессор Хаос»

Описание

Каждый день количество бактерий a преобразуется: $a = a \times b - c$, затем если $a > d$, то $a = d$, и если $a \leq 0$, эксперимент заканчивается. Требуется найти количество бактерий после k -го дня (или 0, если эксперимент завершился раньше).

Ограничение времени: 1 секунда.

Ограничение памяти: 64 Мб.

Решение

Пусть $a_n = ba_{n-1} - c$. Если $a_n < 0$, причем $n < k$, тогда ответ 0. В противном случае будем вычислять значения a_i , пока какое-то из них не повторится. Это случится, причем не позже, чем за d итераций цикла, поскольку $a_i \leq d$. Тогда наши значения a_i зациклились, количество дней которые остались (если к этому моменту эксперимент закончился, то ассимптотика всё та же) делим на длину цикла и находим значение, которое будет в последний день эксперимента (осталось $\leq d \leq 1000$) дней.

Доказательство

На самом деле задача решается за константу. Идея та же, что описана в разделе Решение. Однако алгоритм закончится за не более чем 20 операций. Давайте докажем это.

Пусть $a = a_0$, $\forall n \in \mathbb{N} \ a_n = ba_{n-1} - c$, $\Delta_n = a_n - a_{n-1}$. Заметим, что $\Delta_n = b^{n-1}(a_1 - a_0)$, давайте докажем это по индукции. База очевидна.

$$a_{n-1} = a_0 + \sum_{k=1}^{n-1} \Delta_k = a_0 + (a_1 - a_0)(b^0 + b^1 + \dots + b^{n-2}).$$

$$\Delta_n = a_n - a_{n-1} = (b-1)a_{n-1} - c = (b-1)(a_0 + (a_1 - a_0)(b^0 + b^1 + \dots + b^{n-2})) - c = (b-1)(a_0 + (a_1 - a_0)\frac{b^{n-1}-1}{b-1}) - c = (b^{n-1}-1)(a_1 - a_0) + (b-1)a_0 - c = b^{n-1}(a_1 - a_0), \text{ чтд.}$$

$$a_n = a_0 + \sum_{k=1}^n \Delta_k = a_0 + (a_1 - a_0)(b^0 + b^1 + \dots + b^{n-1}) \geq (a_1 - a_0)\frac{b^n - 1}{b - 1}.$$

Если $b = 1$, то решение очевидно за единицу (последовательность каждый раз уменьшается на c). Пусть $b \geq 2$, заметим также, что $|a_1 - a_0| \geq 1$, значит $|a_n| \geq \frac{b^n - 1}{b - 1}$. Найдём такое b_0 , что при нём минимальное n такое, что $\frac{b^n - 1}{b - 1} \geq d$ максимально. Далее пусть $a_1 > a_0$. Заметим, что $f(x) = \frac{x^n - 1}{x - 1}$ возрастающая функция, значит при росте x минимальное n удовлетворяющее условию уменьшается, значит $b_0 = 2$ и $\log_2(d + 1) \leq 10$ итераций цикла достаточно, чтобы значение a_n стало равным d .

Повторим рассуждения при $a_0 = d$, если $a_1 > d$ тогда a_1 станет равным d , в дальнейшем значения будут равны d . Если же нет, последовательность количества бактерий будет монотонно уменьшаться также по экспоненте.

Если же $a_1 < a_0$, то последовательность убывает с той же скоростью, тогда если дней достаточно, то за не более чем 20 итераций бактерий станет меньше чем c и ответ 0 (последовательность растёт монотонно, так как $\Delta_n = b\Delta_{n-1}$), а если нет, то алгоритм закончится за всё также не более чем 20 итераций.

Алгоритмическая сложность

- По времени: $O(\log_b d)$ короче константа ≈ 20
- По памяти: $O(1)$

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     ios_base::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     long long a;
10    long long b;
11    long long c;
12    long long d;
13    long long k;
14    cin >> a >> b >> c >> d >> k;
15
16    long long count = a;
17
18    for (long long i = 1; i <= k; i++) {
19        long long prev_count = count;
20        count = count * b - c;
21
22        if (count <= 0) {
23            cout << 0;
24            return 0;
25        }
26
27        if (count > d) {
28            count = d;
29        }
30
31        if (count == prev_count) {
32            break;
33        }
34    }
35
36    cout << count;
37    return 0;
38 }
```